

RAG Chatbot — How It Works

A plain-language walkthrough of what this app does and how it's built, written so you can explain it to someone else (or answer questions about it) without re-reading the code.

1. Elevator pitch

You upload documents (PDF, DOCX, TXT, MD). The app reads them, breaks them into small chunks, and turns each chunk into a vector (a list of numbers that captures its meaning). When you ask a question in the chat, it turns *your question* into a vector too, finds the most similar chunks from your documents, and asks an LLM (Google Gemini) to answer using only that retrieved context — not its general knowledge. That's what "RAG" means: **R**etrieval-**A**ugmented **G**eneration. If nothing relevant was retrieved, it's instructed to say so instead of making something up.

2. Tech stack

Layer	Technology
Backend	Laravel 13 (PHP 8.3)
Frontend	React 19 + Tailwind CSS v4, bundled by Vite
Database	SQLite (<code>database/database.sqlite</code>) — stores documents, chunk text, conversations, messages
Vector store	Pinecone — stores only the embedding vectors + light metadata, not the text
LLM (chat)	Google Gemini (<code>gemini-3.5-flash-lite</code>)
LLM (embeddings)	Google Gemini (<code>gemini-embedding-001</code> , truncated to 768 dimensions)

There's no authentication — it's a single shared document/chat space, one instance for whoever's running it locally.

3. Key concept: what's an embedding, and why "768 dimensions"?

This trips people up, so it's worth explaining properly rather than just naming the model.

What `gemini-embedding-001` actually is. It's a Google model, but a very different *kind* of model than the one that writes chat answers (`gemini-3.5-flash-lite`). It doesn't generate text at all — its only job is: take a piece of text in, output a list of numbers. That's it. `embedding` in the name is its purpose; `001` is just Google's version number for this model.

What a "dimension" is. Each number in that output list is one "dimension." A 768-dimension vector is just a list of 768 floating-point numbers, e.g. `[0.0161, -0.0191, 0.0646, ...]` . Think of it like coordinates: a point on a map has 2 numbers (x, y) — 2 dimensions. A point in physical space has 3. An embedding with 768 dimensions is a point in a 768-dimensional space — impossible to visualize, but mathematically the same idea: it's a location.

Why this matters for search. The model is trained so that text with *similar meaning* lands at *nearby points* in that space. "The cat sat on the mat" and "A feline rested on the rug" end up close together; "The stock market crashed" ends up far away. So "find the most relevant chunk" literally becomes "find the nearest point" — that's the entire mechanism behind `VectorStoreService::query()` . More dimensions = more room to capture fine-grained distinctions in meaning, at the cost of more storage and slightly slower comparison.

Why "truncated to 768." `gemini-embedding-001` normally outputs 3072 numbers, not 768. It was trained with a technique (Matryoshka Representation Learning) where the *most important* information is front-loaded into the first numbers of the vector, and each additional number adds progressively finer detail — similar to how a photo thumbnail still clearly shows the subject, just with less fine detail than the full-resolution version. Because of that training approach, you can safely take just the **first 768 of the 3072 numbers** and still get a usable, meaningful vector.

`EmbeddingService.php` asks Gemini for exactly that shortened slice via `outputDimensionality: 768`.

Why 768 specifically, not some other number. It's not a size we chose for performance — it's forced by Pinecone. The vector database's index was originally created for a different (now-retired) embedding model, `text-embedding-004`, which natively outputs 768 numbers. A Pinecone index's dimension is fixed at creation time and can't be changed afterward. So when `text-embedding-004` got retired and we switched to `gemini-embedding-001`, we had to ask it to shrink its output to 768 to stay compatible with the existing index — otherwise every write fails with a "vector dimension mismatch" error.

4. Architecture diagram



5. Flow #1 — Uploading a document

Entry point: `UploadPanel.jsx` → `POST /api/documents` → `DocumentController::store()` → `DocumentIngestionService::ingest()`.

1. **Save the file.** The upload is stored under `storage/app/private/documents/` and a `Document` DB row is created with `status = processing`.
2. **Parse.** `DocumentParserService` extracts plain text — `smalot/pdfparser` for PDFs, `phpoffice/phpword` for DOCX, plain `file_get_contents` for txt/md.
3. **Chunk.** `ChunkerService` splits the text into ~500-character pieces, preferring to break on paragraph boundaries, then sentences, with a small overlap between chunks so context isn't lost at the seam. A 20-page document typically becomes 60-150 chunks.

4. **Embed.** `EmbeddingService::embedMany()` turns every chunk into a vector by calling Gemini's `embedContent` API. This runs **concurrently** (in pools of 10 requests at once via `Http::pool()`) rather than one request at a time — this is what makes a 100-chunk document take ~25-30 seconds instead of several minutes.
5. **Store.** `VectorStoreService::upsertMany()` sends all the vectors to Pinecone in one batched request. At the same time, the actual chunk *text* is bulk-inserted into the local `document_chunks` table. **Pinecone only ever holds the vector + a little metadata (document id, filename, chunk index) — the real text lives in SQLite.** This split exists because vector databases are good at similarity search but bad/expensive at storing large text blobs.
6. The `Document` row is updated to `status = ready` with a `chunk_count`. If anything in steps 2-5 throws, the document is marked `status = failed` with the error message instead of the whole request blowing up.

6. Flow #2 — Asking a question

Entry point: `ChatWindow.jsx` → `POST /api/chat/stream` → `ChatController::askStream()` → `RagChatService::askStream()`.

1. **Embed the question.** The user's message is turned into a vector the same way document chunks are (`EmbeddingService::embed()`).
2. **Retrieve.** `VectorStoreService::query()` asks Pinecone for the 4 most similar chunks (by cosine similarity) across *all* uploaded documents. Their `pinecone_id`s are looked up in SQLite to get back the actual chunk text.
3. **Build the prompt.** The retrieved chunk text becomes the `CONTEXT` block in a system prompt (see `RagChatService::SYSTEM_TEMPLATE`) that instructs Gemini to answer only from that context, keep answers to 2-4 sentences, and say *"I don't have that information in the uploaded documents"* if the context doesn't cover the question. Prior messages in the conversation are also included as history, so follow-up questions work.
4. **Generate, streamed.** `GeminiChatService::generateStream()` calls Gemini's `streamGenerateContent` (Server-Sent Events) endpoint instead of waiting for the full answer. `ChatController` re-emits those SSE events to the browser as they arrive. `ChatWindow.jsx` appends each text fragment to the message bubble live, so the answer appears progressively instead of after a multi-second wait.
5. **Persist.** Once the full answer is assembled, both the user's message and the assistant's answer (plus which sources were used) are saved as `Message` rows, tied to a `Conversation` identified by a `session_uuid` stored in the browser's `sessionStorage` — that's what makes chat history survive a page refresh.

There's also a non-streaming twin of this (`POST /api/chat` → `ChatController::ask()`) that returns the full answer as one JSON response — kept for API consumers that don't want to deal with SSE, but the UI always uses the streaming version.

7. Deleting documents

`DocumentController::destroy()` (one document) and `destroyAll()` (the "Remove all" button) both go through `DocumentIngestionService`, which deletes the matching vectors from Pinecone *first* (best-effort — logged as a warning if Pinecone is briefly unreachable, but doesn't block the delete) and then deletes the `Document` row, which cascades to its `document_chunks` rows via a foreign key.

8. File-by-file reference

Backend (`app/`)

File	Responsibility
<code>Http/Controllers/DocumentController.php</code>	Upload, list, delete one/all documents
<code>Http/Controllers/ChatController.php</code>	Ask a question (JSON or SSE stream), fetch history
<code>Services/DocumentIngestionService.php</code>	Orchestrates parse → chunk → embed → store, and delete
<code>Services/DocumentParserService.php</code>	Extracts raw text from PDF/DOCX/TXT/MD
<code>Services/ChunkerService.php</code>	Splits text into overlapping chunks
<code>Services/EmbeddingService.php</code>	Calls Gemini's embedding API (single + concurrent pool)
<code>Services/VectorStoreService.php</code>	Talks to Pinecone: upsert, query, delete
<code>Services/GeminiChatService.php</code>	Calls Gemini's chat API (single response + SSE stream)
<code>Services/RagChatService.php</code>	The actual RAG logic: retrieve context, build prompt, call the LLM, persist
<code>Models/Document.php</code> , <code>DocumentChunk.php</code> , <code>Conversation.php</code> , <code>Message.php</code>	Eloquent models

Frontend (`resources/js/components/`)

File	Responsibility
<code>App.jsx</code>	Top-level layout, holds the session UUID and document list state
<code>UploadPanel.jsx</code>	Sidebar: upload button, document list with status badges, "Remove all"
<code>ChatWindow.jsx</code>	Message list, the SSE stream consumer, the composer (auto-growing textarea)
<code>SourcesPanel.jsx</code>	Collapsible "which chunks was this answer based on" panel under each answer
<code>Icons.jsx</code>	Shared inline SVG icon set

9. Database schema (SQLite)

- **documents** — `filename` , `original_extension` , `status` (`processing` / `ready` / `failed`), `chunk_count` , `error_message`
- **document_chunks** — `document_id` (FK, cascade delete), `chunk_index` , `content` (the actual text), `pinecone_id` (links to the vector in Pinecone)
- **conversations** — `session_uuid` (ties a browser session to its history)
- **messages** — `conversation_id` (FK, cascade delete), `role` (`user` / `assistant`), `content` , `sources` (JSON, assistant messages only)

10. Common questions

"Where does the AI's answer actually come from?" Only from the 4 most relevant chunks retrieved from your uploaded documents for that specific question, plus the recent conversation history. It's explicitly told not to use outside knowledge.

"What if I ask something not in the documents?" It's instructed to reply *"I don't have that information in the uploaded documents"* rather than guess.

"Does it remember earlier messages in the conversation?" Yes — full message history for that `session_uuid` is sent with every request, so follow-ups work. History is *not* summarized or trimmed, so a very long conversation will slowly get slower/costlier (there's no cap on this today).

"Why does the answer appear word-by-word instead of all at once?" It's streamed from Gemini via Server-Sent Events, so the browser renders each fragment as it arrives instead of waiting for the whole answer.

"Where is my data stored? Is anything sent to a third party?" Document text and chat history live in a local SQLite file. Document text is also transiently sent to Google (Gemini) for embedding/answering, and the resulting vectors (not the raw text) are stored in Pinecone, a hosted vector database.

"What happens when I delete a document?" Its rows are removed from SQLite and its vectors are removed from Pinecone. (The uploaded file itself stays on disk under `storage/app/private/documents/` — there's currently no code path that deletes it, since the stored file path isn't saved anywhere after upload.)

"How long does upload take?" Roughly proportional to chunk count — one Gemini embedding call per chunk (done concurrently, 10 at a time), then a single batched Pinecone write. A ~100-chunk document takes around 25-30 seconds.

"Can I run this without Pinecone/Gemini?" No — both are hard dependencies. `GEMINI_API_KEY`, `GEMINI_CHAT_MODEL`, `GEMINI_EMBEDDING_MODEL`, `PINECONE_API_KEY`, and `PINECONE_HOST` all need to be set in `.env` for anything to work.

"Is there authentication?" No — anyone who can reach the app can see and query every uploaded document. There's no per-user data isolation.